

Atomic Memory Orders: Microarchitectures & Applied Telemetry

In standard multi-threaded C++, the default memory order for any atomic operation is `std::memory_order_seq_cst` (Sequentially Consistent). While safe, sequential consistency enforces a globally agreed-upon total program order across all cores. On modern out-of-order CPUs, this forces the hardware to drain store buffers and insert pipeline memory fences (e.g., `MFENCE` on x86, `DMB ISH` on ARM64).

To achieve zero-overhead instrumentation, our `Telemetry` subsystem bypasses `seq_cst` entirely. Instead, we use explicit, fine-grained memory orders: `relaxed`, `acquire`, and `release`.

1. Hardware Primitives & Memory Order Semantics

To understand what the compiler and CPU are doing, we must look past the C++ syntax into CPU hardware structures: the **Store Buffer**, the **Invalidation Queue**, and the **Out-of-Order Execution Window**.



A. `std::memory_order_relaxed`

- **Guarantee:** Only **atomicity** and **modification order consistency** on the single variable itself. It guarantees that operations on *this specific variable* will never be torn or duplicated, and all threads see values progress monotonically.
- **Zero Synchronization:** It establishes **no synchronization** or happens-before relationship with any *other* variable or memory address.
- **Hardware Implementation:**
 - **x86-64:** Plain load (`mov`) or store (`mov`). For atomic arithmetic, it is a single `lock add` OR `lock inc`. It emits **zero fence instructions**.
 - **ARM64:** Simple atomic primitives (`ldadd`, `stadd` in ARMv8.1+ LSE, or standard `ldr` / `str` with no barriers).
- **Pipeline Cost:** Near zero. The CPU's out-of-order engine is free to reorder reads and writes around this operation across independent addresses.

B. `std::memory_order_release`

- **Guarantee: One-way write barrier.** No reads or writes in the current thread that appear *before* this store in source code can be reordered *after* this store by the compiler or the hardware.
- **Hardware Implementation:**
 - **x86-64:** Emitted as a regular `mov`. Because x86 has a Total Store Order (TSO) memory model, stores are never reordered with older stores or older reads. Release is architectural hardware behavior on x86 for free.
 - **ARM64:** Generates a store-release instruction (`stlrr`), which forces the core's local store buffer to commit prior memory modifications before the release store becomes globally visible to other observers.

C. `std::memory_order_acquire`

- **Guarantee: One-way read barrier.** No reads or writes in the current thread that appear *after* this load in source code can be reordered *before* this load by the compiler or the hardware.
- **Hardware Implementation:**
 - **x86-64:** Emitted as a regular `mov`. Loads are not reordered with subsequent loads or stores on x86. Free at the instruction level.
 - **ARM64:** Generates a load-acquire instruction (`ldar`), which prevents the CPU's speculative execution engine from consuming downstream memory loads until this load completes and coherency resolves.

D. The Acquire-Release Synchronization Pair

When Thread A performs a **Release Store** on variable X , and Thread B performs an **Acquire Load** on the same variable X and observes the written value:

All memory writes made by Thread A before the release store → **Synchronize**

2. Telemetry Applied Case Studies

Let us dissect the exact usage of atomic operations throughout the `Telemetry` class.

Case 1: The High-Frequency Metric Counter

Used in `count()`, `add()`, and `should_report()`:

```
// Hot Path: Recording a metric event
slots[slot].metrics[index(m)].count.fetch_add(1, std::memory_order_relaxed);
```

Why `memory_order_relaxed`?

- **Problem to Solve:** We need to increment an accumulator hundreds of millions of times across threads with minimal CPU cycle overhead.
- **Analysis:** We are tracking independent, statistical counters. Thread A incrementing `count` does not gate, signal, or guard any other payload data in memory. Thread B does not need to synchronize its pipeline state with Thread A's execution history just to read an aggregated number.
- **Microarchitectural Flow:**
 1. Core executes `lock add [slot_metric_count], 1`.
 2. Because each thread owns its own cache line (`alignas(64)`), this cache line already lives in the core's private L1d cache in the **Modified (M)** state.
 3. The operation completes entirely inside the local execution unit and store buffer. Zero bus transactions occur. Zero memory barriers are dispatched.

Case 2: The Two-Variable Tag Publication Pattern

Used in `set_tag()` and `find_by_tag()`:

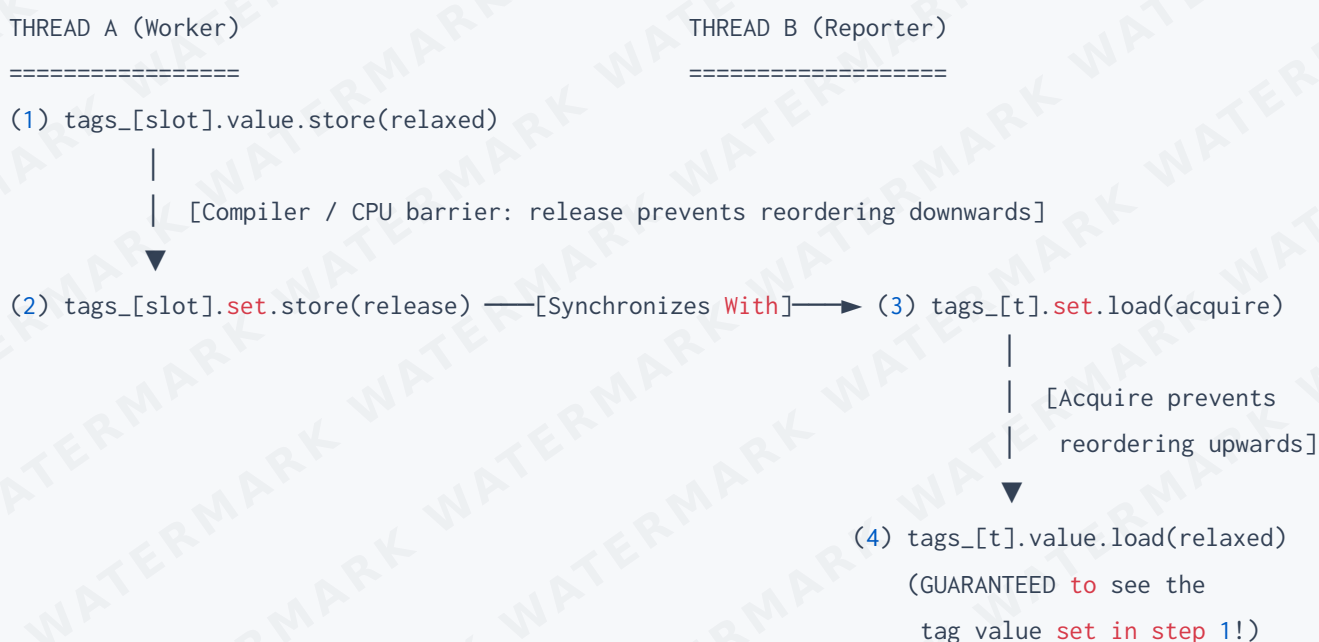
```
// Writer Thread (Worker)
inline void set_tag(std::uint64_t tag) noexcept {
    const std::size_t slot = get_thread_slot();
    tags[slot].value.store(tag, std::memory_order_relaxed); // (1)
    tags[slot].set.store(true, std::memory_order_release);  // (2)
}
```

```

// Reader Thread (Reporter)
static bool find_by_tag(std::uint64_t tag, Report& out) noexcept {
    // ...
    for (std::size_t t = 0; t < MaxThreads; ++t) {
        if (!self.tags_[t].set.load(std::memory_order_acquire)) continue; // (3)
        if (self.tags_[t].value.load(std::memory_order_relaxed) != tag) // (4)
            continue;
        // ...
    }
}

```

Mechanical Dissection:



Why Not Relaxed for Both?

If step (2) used `relaxed` instead of `release`, an out-of-order processor (like ARM64 or PowerPC) or an optimizing compiler could reorder the execution of stores:

Step 2 (flag store = true) could become visible **BEFORE** Step 1 (value store)

If the reporter thread reads `tags[t].set == true` at step (3) under relaxed ordering, it could proceed to step (4) and load `tags[t].value` while it still contains the uninitialized default value `0`!

By combining `store(..., release)` with `load(..., acquire)`, we create a memory synchronization edge. Any write that happens before step (2) is guaranteed to be visible to

any thread that reads step (3) and sees `true`.

Case 3: The Compare-And-Swap (CAS) Extrema Loops

Used in `atomic_min()` and `atomic_max()`:

```
static void atomic_min(std::atomic<std::int64_t>& target, std::int64_t val) noexcept {
    std::int64_t current = target.load(std::memory_order_relaxed);
    while (val < current && !target.compare_exchange_weak(
        current, val,
        std::memory_order_relaxed,
        std::memory_order_relaxed))
    {
        // Loop body intentionally empty: current is updated with latest target on failure
    }
}
```

Why `compare_exchange_weak`?

- `compare_exchange_strong` guarantees that it will only return `false` if the target memory value actually differed from `current`. To provide this guarantee on LL/SC (Load-Linked / Store-Conditional) architectures (ARM64, RISC-V), the CPU must generate loops to catch spurious context-switch failures.
- `compare_exchange_weak` is permitted to fail spuriously (e.g., due to an interrupt or cache line eviction) even if the values match.
- The Telemetry Rule:** Because we are **already wrapping the CAS operation inside a loop** (`while(...)`), using `compare_exchange_weak` generates the absolute minimal instruction sequence on all CPU targets without generating nested loops in hardware.

Why `relaxed` for Both Success and Failure?

We are calculating mathematical extrema across scalar numeric values (`min` / `max` bounds). Updating an extremum does not act as a signal flag to unlock or publish another data structure. Therefore, synchronization barriers are unnecessary:

```
target.compare_exchange_weak(
    current, val,
    std::memory_order_relaxed, // Success order: don't inject fences
    std::memory_order_relaxed // Failure order: reload 'current' without fences
);
```

On an x86 processor, this compiles down to the single instruction:

```
lock cmpxchg qword ptr [rdi], rsi
```

The `lock` prefix ensures cache coherency across cores, but avoids redundant pipeline serialization fences.

Case 4: Dynamic Thread-Slot Claiming

Used in `get_thread_slot()`:

```
bool expected = false;
if (self.slot_used[i].compare_exchange_strong(
    expected, true,
    std::memory_order_relaxed))
{
    return i;
}
```

The Mechanical Life Cycle:

- 1. **Infrequent Path:** This executes **only once** per thread lifetime on initial kernel invocation. The claimed slot index is immediately stored in `thread_local std::size_t slot_idx`.
- 2. **Semantic Needs:** The slot reservation only needs to be atomic to ensure no two threads claim the same slot. Once `expected == false` succeeds, slot ownership is transferred exclusively to the calling thread for its entire lifespan.
- 3. **Why Relaxed is Safe:** Thread ownership of a slot does not publish any shared dataset to other threads yet. The thread will begin recording to `slots[i]` in subsequent instructions.
- 4. **Why strong over weak here?** Unlike our `atomic_min` loop, here we are iterating through an array (for (std::size_t i = 0; i < MaxThreads; ++i)). If we used `weak`, a spurious failure would cause us to unnecessarily skip an open slot and test the next one. We want an explicit, deterministic CAS failure before evaluating slot $i + 1$.

3. Atomic Operations Reference Table

Function	Operation	Memory Order	Hardware Role
<code>count()</code>	<code>fetch_add(1)</code>	<code>relaxed</code>	Local cache-line atomic counter. Zero bus serialization.

Function	Operation	Memory Order	Hardware Role
<code>add()</code>	<code>fetch_add(fv)</code>	<code>relaxed</code>	Scaled 64-bit integer accumulation. Zero pipeline fences.
<code>atomic_min</code>	<code>compare_exchange_weak</code>	<code>relaxed</code>	Lock-free CAS loop. Minimum instruction footprint.
<code>set_tag()</code>	<code>store(tag)</code>	<code>relaxed</code>	Stores the payload before the publish signal.
<code>set_tag()</code>	<code>store(true)</code>	<code>release</code>	Publish boundary. Ensures payload is globally visible first.
<code>find_by_tag()</code>	<code>load()</code>	<code>acquire</code>	Consume boundary. Validates signal before reading payload.
<code>collect_all()</code>	<code>load()</code>	<code>relaxed</code>	Snapshot aggregator. Reads dirty or modified metrics on-the-fly.
<code>clear()</code>	<code>store(0)</code>	<code>relaxed</code>	Maintenance reset. Does not synchronize downstream work.

Summary

By mapping our memory orders directly to hardware realities:

1. **Counters** consume minimal cycles via `memory_order_relaxed`, hitting pure L1 cache.
2. **Payload publications** (`tag_value` guarded by `tag_set`) are kept race-free and cross-architecture portable using explicit `acquire` / `release` pairs.
3. **Extrema checks** leverage hardware-friendly `compare_exchange_weak` loops with relaxed ordering.
4. **Heavy barriers** (`seq_cst` , `MFENCE`) are eliminated from the kernel hot path, keeping our execution latency identical to non-atomic code.